Mid-Term Report

Seasons of Code Project

Competitive Programming from First Principles

Submitted By

Anjali Jogi

Mentors

Nishkarsh, Harshit

Institution

Indian Institute of Technology Bombay

Mid-Term Progress Report

2026

Contents

Introduction

Competitive Programming is a problem-solving discipline that focuses on developing logical thinking, algorithmic reasoning, and efficient implementation skills. Unlike traditional programming assignments, competitive programming requires participants to analyze unfamiliar problems, identify patterns, reason about constraints, and produce efficient solutions within limited time.

The objective of this Summer of Code project, *Competitive Programming from First Principles*, is to build these skills gradually through a structured learning path. Rather than beginning with advanced algorithms, the project starts with the fundamentals and progressively develops the ability to think like a competitive programmer.

During the first four weeks, the emphasis was placed on understanding core concepts, practicing implementation, and solving problems across different online judges. The learning process involved studying theoretical ideas, observing how those ideas are applied in problems, and then solving a variety of exercises to reinforce understanding.

The project has focused not only on learning techniques but also on developing a systematic approach to problem solving. Throughout the first half of the project, increasing attention was given to identifying patterns, analyzing complexity, understanding observations, and constructing efficient solutions.

This report summarizes the concepts studied, the practical work completed, and the major learning outcomes achieved during the first four weeks of the project.

Project Objectives

The primary objectives of this project are:

- Develop strong problem-solving skills.
- Build a solid foundation in competitive programming.
- Improve logical and analytical thinking.
- Learn to analyze constraints and evaluate efficiency.
- Strengthen implementation skills using C++.
- Develop familiarity with contest-style problems.
- Learn how to derive solutions through observations and reasoning.
- Build consistency through regular problem-solving practice.

The long-term goal is to become capable of approaching unfamiliar problems systematically and constructing efficient solutions independently.

Week 1: Building the Foundations

The first week focused on establishing the basic concepts required for competitive programming. Before attempting challenging contest problems, it is important to understand the tools, techniques, and analytical methods that form the foundation of efficient problem solving.

The topics covered during this week were:

- C++ Standard Template Library (STL)
- Time Complexity Analysis
- Prefix Sums
- Difference Arrays

C++ Standard Template Library

The week began with a revision of the C++ Standard Template Library (STL), which is one of the most important resources available to competitive programmers.

Instead of implementing common data structures repeatedly from scratch, STL provides optimized and well-tested implementations that can be used directly during contests.

The revision included commonly used containers and utilities such as:

- Vectors
- Pairs
- Sets
- Maps
- Queues
- Stacks
- Priority Queues
- Sorting Functions

A major focus during this stage was understanding when each data structure should be used and what performance characteristics it provides.

For example, vectors allow efficient storage and traversal of elements, while sets and maps maintain ordered data and support efficient insertion and lookup operations.

Understanding these tools early was important because they appear frequently in competitive programming problems and often simplify implementation considerably.

Time Complexity Analysis

The next topic covered was time complexity analysis.

One of the key lessons learned during this stage was that solving a problem correctly is not always sufficient. A solution must also be efficient enough to run within the given constraints.

Time complexity provides a way to estimate how the running time of an algorithm grows as the input size increases.

The following complexity classes were studied:

- Constant Time — $O(1)$
- Logarithmic Time — $O(\log n)$
- Linear Time — $O(n)$
- Linearithmic Time — $O(n \log n)$
- Quadratic Time — $O(n^2)$

Through examples and practice problems, it became clear that choosing an efficient algorithm often matters more than implementation details.

This topic also introduced the habit of analyzing constraints before writing code. Instead of immediately implementing an idea, it became important to first estimate whether the proposed solution would be feasible for the given input size.

Prefix Sums

Prefix sums were introduced as one of the first optimization techniques encountered during the project.

The fundamental idea behind prefix sums is to preprocess information so that repeated calculations can be performed more efficiently.

Without preprocessing, answering multiple range-sum queries may require repeatedly traversing portions of an array. Prefix sums eliminate this repeated work by storing cumulative information in advance.

Studying prefix sums helped develop an understanding of how preprocessing can transform a problem that appears expensive into one that can be solved efficiently.

Important learning outcomes included:

- Constructing prefix sum arrays.
- Understanding cumulative information.
- Answering range queries efficiently.

- Reducing unnecessary repeated computation.

Beyond the technique itself, this topic demonstrated a broader problem-solving principle: investing time in preprocessing can significantly improve overall performance.

Difference Arrays

Difference arrays were studied as a complementary concept to prefix sums.

While prefix sums are useful for answering range queries efficiently, difference arrays help perform range updates efficiently.

The key idea is to record changes rather than updating every element individually. Once all updates have been processed, the final values can be reconstructed using cumulative calculations.

Initially, the concept appeared unintuitive because it required thinking about changes rather than actual values. However, after working through examples, the advantages became clear.

The technique demonstrated how a different representation of information can simplify a problem considerably.

Major takeaways included:

- Efficient handling of multiple updates.
- Understanding cumulative changes.
- Relationship between difference arrays and prefix sums.
- Reduction of unnecessary computations.

Practice and Problem Solving

A significant portion of Week 1 was devoted to implementation practice.

Mandatory problems from the provided sheets were solved, covering topics such as:

- Arithmetic operations
- Conditional logic
- Mathematical computations
- Input and output handling
- Basic implementation techniques

Although these problems were not algorithmically complex, they played an important role in improving coding speed, accuracy, and familiarity with online judges.

Additional problems from the practice sheets were also attempted to strengthen understanding and gain confidence.

Key Learnings from Week 1

Week 1 established the foundation upon which the remainder of the project would build.

The most important lessons learned were:

- Efficient programming requires understanding complexity.
- STL provides powerful tools that simplify implementation.
- Preprocessing techniques can dramatically improve performance.
- Problem constraints should influence solution design.
- Consistent practice is essential for improving implementation skills.

The concepts studied during this week provided the necessary groundwork for tackling more challenging problem-solving tasks in the following weeks.

Week 2: Developing Competitive Programming Intuition

After building familiarity with fundamental concepts during the first week, the focus of Week 2 shifted toward actual competitive programming problem solving.

Unlike Week 1, where the primary objective was understanding tools and techniques, Week 2 emphasized developing the thought process required to solve contest-style problems.

A major theme throughout the week was understanding that many beginner and intermediate competitive programming problems are not solved using sophisticated algorithms. Instead, they often depend on observations, logical reasoning, pattern recognition, and careful implementation.

The topics and practice areas covered during this week included:

- Ad-hoc Problem Solving
- Constructive Thinking
- Prefix Sum Applications

- Frequency Counting
- Sorting-Based Problem Solving
- Introduction to Codeforces Div 2 A/B Problems

Understanding Ad-hoc Problem Solving

One of the most important lessons from Week 2 was learning how to approach ad-hoc problems.

Unlike algorithm-based questions where a known technique can be applied directly, ad-hoc problems require discovering the solution through observations and logical reasoning.

Initially, these problems felt challenging because there was no obvious formula or algorithm to follow. The solution often depended on noticing a hidden pattern or understanding a special property of the problem.

Through repeated practice, a structured approach gradually emerged:

1. Read the statement carefully.
2. Understand the constraints.
3. Create small examples manually.
4. Observe patterns and relationships.
5. Formulate a possible solution.
6. Verify the idea using additional examples.
7. Implement and test the solution.

Developing this approach was one of the most valuable outcomes of the week because it encouraged independent thinking rather than memorization.

Constructive Thinking

Another important aspect of Week 2 was learning constructive thinking.

Many competitive programming problems require constructing a valid arrangement, sequence, or configuration that satisfies a set of conditions.

These problems differ from traditional computational tasks because the objective is not merely to calculate a value but to design a valid solution.

Working on such problems highlighted the importance of:

- Experimenting with small cases.

- Identifying recurring patterns.
- Looking for simple constructions.
- Verifying whether all constraints are satisfied.

This type of problem solving encouraged creativity and strengthened the ability to reason about conditions systematically.

Applications of Prefix Sums

While prefix sums were introduced during Week 1, Week 2 focused on applying them to practical problem-solving scenarios.

Several problems demonstrated how preprocessing cumulative information can simplify otherwise expensive computations.

Through these exercises, it became evident that many problems involving ranges, cumulative values, and subarrays become significantly easier after transforming the original data into prefix sums.

Important observations gained during practice included:

- Preprocessing can reduce repeated work.
- Range-based queries often benefit from cumulative information.
- A suitable representation of data can simplify implementation.
- Optimization frequently begins with identifying redundant calculations.

This reinforced the broader idea that problem solving is often about representing information in a useful way.

Frequency Counting Techniques

Frequency counting emerged as another recurring theme throughout the week.

Several problems required tracking how often values appeared and using that information to make decisions.

Instead of repeatedly scanning entire arrays, frequency information can be maintained efficiently and reused whenever necessary.

Working with frequency-based approaches improved understanding of:

- Counting occurrences efficiently.
- Identifying duplicates and repetitions.

- Tracking information across large datasets.
- Simplifying implementation through appropriate data structures.

An important takeaway was recognizing situations where storing additional information can significantly reduce computational effort.

Sorting as a Problem-Solving Tool

Week 2 also highlighted the usefulness of sorting.

Initially, sorting may appear to be a preprocessing step rather than a solution technique. However, many practice problems demonstrated that sorting often reveals useful structure within the data.

After sorting, relationships between elements become easier to analyze, making it possible to identify observations that would otherwise remain hidden.

Through problem-solving practice, several advantages of sorting became apparent:

- Easier comparison between values.
- Improved organization of data.
- Simplification of implementation.
- Discovery of useful patterns and observations.

This experience helped develop the habit of considering sorting as a potential first step when approaching new problems.

Practice Through CSES Problems

A major component of Week 2 involved solving carefully selected problems from the CSES Problem Set.

The problems covered a variety of ideas including subarray processing, counting techniques, and sorting-based observations.

Examples included:

- Subarray Sums I
- Subarray Sums II
- Subarray Divisibility
- Distinct Numbers
- Apartments

- Ferris Wheel
- Concert Tickets
- Restaurant Customers
- Movie Festival
- Sum of Two Values

These problems served as an effective bridge between theoretical learning and practical implementation.

They also provided exposure to the style of reasoning commonly required in competitive programming contests.

Beginning Codeforces Practice

Another significant milestone during Week 2 was beginning systematic practice on Codeforces.

The primary objective was not to solve highly difficult problems immediately but rather to become comfortable with contest-style problem statements and develop consistency.

Practice initially focused on lower-rated problems.

This stage was valuable because it provided opportunities to:

- Apply recently learned concepts.
- Improve implementation speed.
- Gain confidence through consistent solving.
- Learn how contest problems are structured.
- Develop familiarity with common patterns.

Repeated exposure to Codeforces problems gradually reduced hesitation when approaching unfamiliar questions.

Major Learning Outcomes

By the end of Week 2, several important improvements were noticeable.

- Increased confidence while approaching unfamiliar problems.
- Better ability to identify useful observations.

- Improved understanding of how constraints influence solutions.
- Greater familiarity with contest-style thinking.
- Enhanced implementation and debugging skills.
- Improved patience while working through difficult problems.

Perhaps the most important lesson from this week was that competitive programming is not solely about learning algorithms. Success often depends on the ability to think critically, identify patterns, and reason about problems systematically.

The experience gained during Week 2 provided the foundation for studying more structured problem-solving paradigms in the following weeks.

Week 3: Greedy Algorithms and Binary Search

After spending the previous week developing observation-based problem-solving skills, Week 3 introduced two of the most frequently used paradigms in competitive programming: Greedy Algorithms and Binary Search.

Unlike many ad-hoc problems where the solution emerges primarily through observation, these topics provide more structured approaches that can be applied across a wide variety of problems.

The objective of this week was not only to learn the techniques themselves but also to understand the reasoning behind them. A major emphasis was placed on identifying situations where these approaches are applicable and understanding why they work.

The topics covered during this week were:

- Greedy Algorithms
- Exchange Arguments
- Binary Search
- Binary Search on Answer
- Searching and Sorting Practice

Introduction to Greedy Algorithms

Greedy algorithms are based on the idea of making the best possible decision at the current step with the hope that these local decisions lead to an overall optimal solution.

At first, this approach may seem risky because choosing the best option immediately does not always guarantee the best final answer. Through examples and practice, it

became clear that greedy strategies only work when specific properties are present within a problem.

A significant portion of the learning process involved understanding the difference between:

- Problems where greedy approaches succeed.
- Problems where greedy approaches fail.

This distinction was important because blindly applying greedy reasoning can easily lead to incorrect solutions.

Studying greedy algorithms encouraged careful analysis of problem structure before deciding on an approach.

Developing Greedy Intuition

One of the main goals of the week was learning how to recognize situations where greedy thinking might be useful.

Rather than memorizing individual solutions, emphasis was placed on identifying common characteristics of greedy problems.

Through problem-solving practice, several useful habits were developed:

- Looking for choices that appear naturally optimal.
- Examining whether local decisions affect future options.
- Testing ideas on small examples.
- Searching for counterexamples that break a proposed strategy.

This process helped build intuition and improved confidence when evaluating potential greedy approaches.

Exchange Arguments

An important theoretical concept introduced during this week was the exchange argument.

While a greedy solution may appear correct intuitively, it is often necessary to justify why the chosen strategy always produces an optimal answer.

Exchange arguments provide a framework for reasoning about correctness.

The general idea is to compare an optimal solution with the greedy solution and show that replacing certain decisions does not make the solution worse. By repeatedly

performing such exchanges, it becomes possible to transform an optimal solution into the greedy solution.

Although the formal proofs can be challenging, studying exchange arguments highlighted the importance of reasoning about correctness rather than relying solely on intuition.

This topic encouraged a deeper understanding of why algorithms work instead of simply accepting them as valid.

Binary Search Fundamentals

The second major topic of the week was Binary Search.

Prior to this stage, binary search was often viewed simply as a method for finding an element within a sorted array.

However, Week 3 demonstrated that binary search is a much broader technique that can be applied whenever a search space possesses a suitable structure.

The following concepts were studied:

- Search boundaries.
- Midpoint calculation.
- Dividing the search space efficiently.
- Maintaining correctness throughout the search process.
- Handling edge cases.

A key realization was that binary search dramatically reduces the number of operations required to locate an answer.

Instead of checking every possibility individually, large portions of the search space can be eliminated at each step.

Binary Search on Answer

One of the most important concepts introduced during the week was Binary Search on Answer.

Many competitive programming problems require finding the smallest or largest value that satisfies a particular condition.

Rather than directly computing the answer, it is often possible to:

1. Guess a candidate answer.
2. Check whether it satisfies the required condition.

3. Use the result to eliminate part of the search space.

4. Repeat until the optimal answer is found.

Initially, this approach felt unintuitive because the search is performed on possible answers rather than on actual data.

However, after solving multiple examples, it became clear why this technique is so powerful.

The most important insight gained from this topic was the concept of monotonicity. When a condition changes from false to true (or true to false) in a predictable manner, binary search can often be applied effectively.

Understanding this idea significantly expanded the range of problems that could be approached confidently.

Searching and Sorting Practice

To reinforce the concepts learned during the week, extensive practice was carried out using the Searching and Sorting section of the CSES Problem Set.

These problems provided opportunities to apply:

- Greedy observations.
- Efficient searching strategies.
- Sorting-based reasoning.
- Complexity analysis.

Repeated exposure to such problems helped improve pattern recognition and strengthened the ability to identify suitable approaches quickly.

The practice also highlighted how multiple concepts often interact within a single problem.

Codeforces Practice

Additional practice was performed through Codeforces problems filtered using the recommended tags and rating ranges.

These problems offered valuable opportunities to apply newly learned techniques in contest-style environments.

During this process, greater attention was given to:

- Reading statements carefully.

- Identifying observations before coding.
- Evaluating complexity before implementation.
- Testing solutions against edge cases.

The experience reinforced the importance of balancing theoretical understanding with practical implementation skills.

Major Learning Outcomes

Week 3 represented an important transition from observation-driven problem solving toward structured algorithmic thinking.

The most significant outcomes of the week included:

- Understanding the core principles of greedy algorithms.
- Learning how exchange arguments justify correctness.
- Developing intuition for evaluating greedy strategies.
- Expanding the understanding of binary search beyond sorted arrays.
- Learning the concept of Binary Search on Answer.
- Appreciating the role of monotonicity in optimization problems.
- Improving confidence when approaching medium-difficulty contest problems.

Overall, the concepts covered during this week provided powerful tools that are widely applicable across competitive programming and laid the groundwork for tackling more advanced problems in the future.

Week 4: Bit Manipulation and Basic Number Theory

The fourth week focused on two important areas that appear frequently in competitive programming: Bit Manipulation and Basic Number Theory.

Both topics provide efficient ways to solve problems that may appear difficult when approached using straightforward methods. While bit manipulation makes use of the binary representation of numbers, number theory focuses on mathematical properties that can simplify computations and reveal hidden patterns.

The goal of this week was to develop familiarity with these concepts and understand how they can be applied during problem solving.

The topics covered were:

- Basic Bit Manipulation
- XOR Operations
- Binary Representation of Numbers
- Basic Number Theory
- Problem Solving Using the Above Concepts

Introduction to Bit Manipulation

Bit manipulation involves working directly with the binary representation of integers.

At the beginning, this topic felt different from previous concepts because it required thinking about numbers in terms of bits rather than their decimal values.

The following bitwise operations were studied:

- AND
- OR
- XOR
- Left Shift
- Right Shift

Understanding these operations required developing familiarity with binary representations and observing how individual bits contribute to the value of a number.

One important lesson from this topic was that many operations can be performed very efficiently when binary properties are utilized correctly.

Understanding XOR

A significant portion of the week was devoted to understanding XOR because of its frequent appearance in competitive programming problems.

Several useful properties of XOR were studied and applied during problem solving.

Through examples and practice, it became clear that XOR possesses patterns that can simplify problems considerably.

Working with XOR also reinforced the importance of looking for mathematical observations rather than relying solely on brute-force computation.

Many solutions became easier after identifying the appropriate XOR property and applying it correctly.

Binary Thinking in Problem Solving

Another important learning outcome was becoming comfortable with binary representations.

Initially, it was natural to think only in decimal numbers. However, bit manipulation problems often become simpler when viewed from a binary perspective.

Practice during this week helped develop the habit of:

- Examining the binary form of numbers.
- Looking for patterns among bits.
- Understanding how operations affect individual positions.
- Relating binary observations to problem constraints.

This shift in perspective proved useful when solving several assigned problems.

Bit Manipulation Practice

The assigned bit manipulation problems provided opportunities to apply theoretical concepts in practical situations.

Problems such as:

- XORwice
- Raising Bacteria
- AND 0, Sum Big
- You Are Given Two Binary Strings
- Johnny and His Hobbies
- ANDSUBAR
- Co-growing Sequence

demonstrated different ways in which binary representations can be used to simplify reasoning and improve efficiency.

Through these exercises, confidence in working with bitwise operations increased significantly.

Introduction to Basic Number Theory

The second major topic covered during Week 4 was Basic Number Theory.

The objective was not to study advanced mathematical techniques but rather to build a strong understanding of the fundamental concepts required for competitive programming.

The learning resources focused on introductory number theory topics and provided insight into how mathematical reasoning can be used to solve computational problems efficiently.

A major realization during this stage was that many problems become easier after identifying the mathematical structure hidden within them.

Mathematical Reasoning in Competitive Programming

Number theory introduced a different style of thinking compared to previous weeks.

Instead of focusing primarily on implementation, greater emphasis was placed on identifying useful mathematical observations.

Through practice, it became clear that:

- Understanding number properties can simplify solutions.
- Mathematical observations often reduce computational effort.
- Pattern recognition plays an important role in problem solving.
- Efficient solutions frequently emerge from mathematical insights.

This experience reinforced the idea that competitive programming requires both programming ability and mathematical reasoning.

Number Theory Practice

Several assigned problems were solved to strengthen understanding of the concepts covered during the week.

These problems involved applying mathematical observations, reasoning about numbers, and designing efficient implementations.

Examples included problems from:

- LeetCode
- AtCoder
- Codeforces

- SPOJ

Working through these problems helped improve confidence when dealing with mathematically oriented questions.

Major Learning Outcomes

The most important outcomes of Week 4 were:

- Understanding binary representations more effectively.
- Becoming comfortable with bitwise operations.
- Learning how XOR can simplify problem solving.
- Developing a stronger mathematical approach to programming problems.
- Improving the ability to identify useful patterns.
- Strengthening analytical thinking through number theory practice.

The concepts studied during this week expanded the range of problems that could be approached confidently and provided valuable tools for future learning.

Conclusion

The first four weeks of the Summer of Code project *Competitive Programming from First Principles* have provided a strong introduction to the fundamental ideas required for competitive programming.

The journey began with essential concepts such as STL, time complexity, prefix sums, and difference arrays. It then progressed toward observation-based problem solving, constructive thinking, sorting techniques, greedy algorithms, binary search, bit manipulation, and basic number theory.

An important outcome of this phase has been the development of a more systematic approach to problem solving. Rather than immediately attempting implementation, greater emphasis is now placed on understanding constraints, identifying patterns, evaluating efficiency, and reasoning about possible solutions.

The combination of theoretical learning and consistent practice has strengthened both programming ability and analytical thinking. The knowledge gained during these four weeks provides a solid foundation for future progress in competitive programming.

References

1. Competitive Programmer's Handbook
2. CSES Problem Set
3. Codeforces Problem Set
4. Codeforces EDU Resources
5. Week 1 Resource Material (STL, Time Complexity, Prefix Sums, Difference Arrays)
6. Week 2 Resource Material (Problem Solving and Codeforces Progression)
7. Week 3 Resource Material (Greedy Algorithms and Binary Search)
8. Week 4 Resource Material (Bit Manipulation and Basic Number Theory)
9. LeetCode Practice Problems